Physics Hand

Physics based hands for Unity that interact with and conform to the world naturally.

V 1.2.0

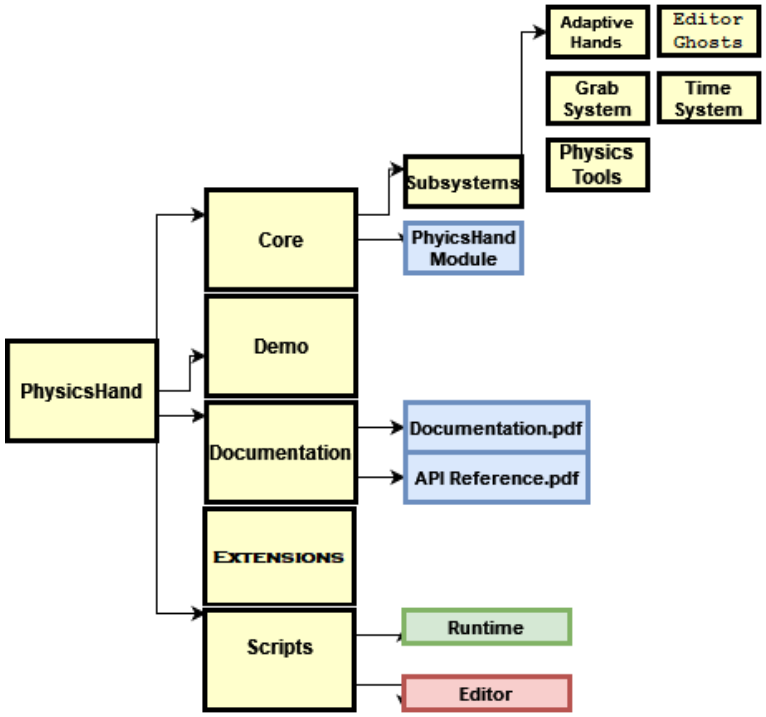
- Incomplete documentation parts will be improved over time.
- Get the most up to date documentation by clicking here.
- Remember you can hover over fields in the “Inspector” window in Unity’s editor to read tooltip explanations of each field.
- If you have any questions or need assistance email support at intuitivegamingsolutions@gmail.com.

Table of Contents

NOTE: See 'API Reference.pdf' (online) if you are looking for source code documentation for the 'Physics Hand' core module.

Folder Structure Overview

A quick overview of the contents of the 'PhysicsHand' directory.



A flowchart showing a partial hierarchy for the 'PhysicsHand' directory.

- **Core** - A folder that contains all of the core components that make up 'Physics Hand'.
  - Subsystems - A folder that contains all of the subsystems that make up 'Physics Hand'.
    - Adaptive Hands - The module responsible for the visual kinematic hand control.
    - Grab System - The module responsible for grabbing and throwing.
    - Physics Tools - A module that provides useful collision, rigidbody, velocity tracking, and more physics tools.
    - Time System - The module that provides the current time.
    - Editor Ghosts - The module that is responsible for creating and managing ghost representations of objects.
  - PhysicsHand Module - The module that ties all of the subsystems together to make a realistic physical hand.
- **Demo** - A folder that contains all demo content for 'Physics Hand'.
- **Documentation** - A folder that contains the documentation for 'Physics Hand'.
  - Documentation.pdf - The hand-written documentation with specific instructions, FAQ, and more.
  - API Reference.pdf - The automatically generated API scripting reference. Useful for coders.
- **Extensions** - A folder that contains all extra/optional scripts.
- **Scripts** - A folder that contains all extra, required scripts that may need to be changed on a game-to-game basis..
  - Runtime - Any optional script that is executed at runtime.
  - Editor - Any optional script that is executed in the editor.

- ***Note:*** The 'Physics Hand' module depends on all of the subsystems however none of the subsystems depend on the 'Physics Hand' module (they may depend on each other). The subsystems are required for 'Physics Hand' to function but the subsystems may be used in smaller projects if their functionality is useful.

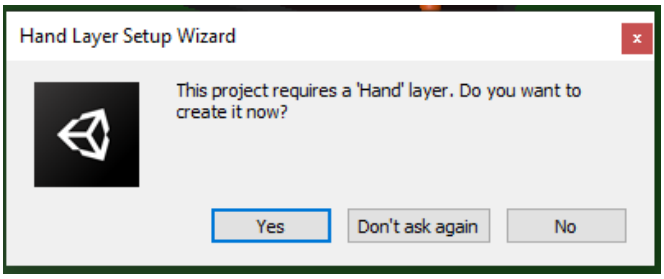
Getting Started

Check out this 'Youtube Walkthrough' for a quick walkthrough on how to get started.

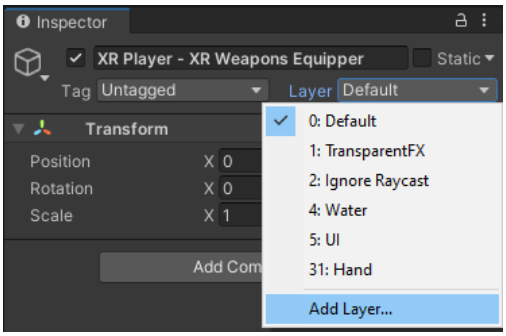
3.a. IMPORTANT: Importing & Setting Up Layers

3.a.i. Before Importing 'Physics Hand'

- This step can often be skipped by using the built-in layer setup wizard that pops up when you have no 'Hand' layer configured.



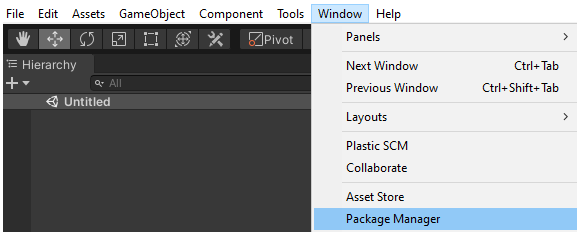
- It is important that the first step you take when setting up 'Physics Hand' is setting up layers. By default with any of the demo physics hands their finger bending will be prevented by self-collisions.
- This ***must be done manually*** because we do not allow the asset to make breaking changes to your project.
- Here's what you need to do:
    1. Open the 'Layer' dropdown in the Unity Editor and select 'Add Layer'.



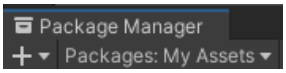
    2. Add a new layer for the 'Hand', you may name it whatever you like and put it in whatever layer slot you want.

3.a.ii. Importing 'Physics Hand'

- The next step is to import Physics Hand. To do this either:
- Navigate to 'Windows → Package Manager' in the Unity Editor toolbar.



- Go to 'My Assets'



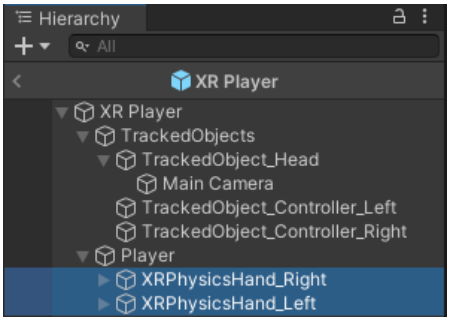
- Search for 'Physics Hand' in the search bar, select the package, click 'Download' (if not already downloaded) and then click 'Import'.

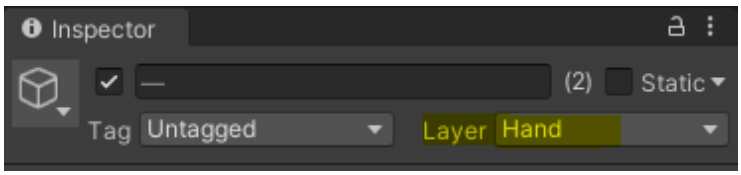
- Simply click 'Import' on the package importer window that pops up and let Unity install 'Physics Hand' in your project.

3.a.iii. After Importing 'Physics Hand'

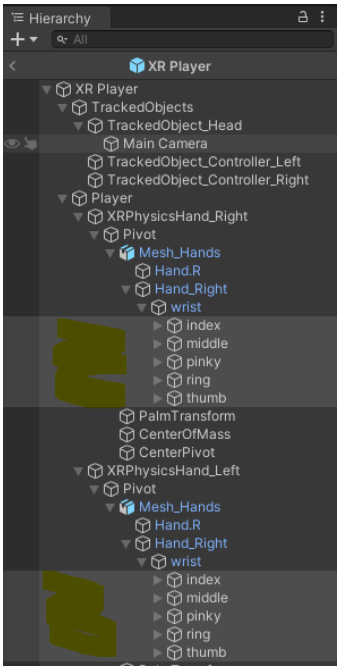
- ***The following steps are only necessary if you used a Layer # other than 31 for the 'Hand' layer.***
- The following step(s) are best performed after setting up 'Physics Hand' so that existing demo assets are able to find the 'Hand' layer.

3. Open the 'XR Player' prefab, select both of the 'XRPhysicsHand_…' GameObjects from the 'Hierarchy' and change their and their children's layers to your new hand layer.

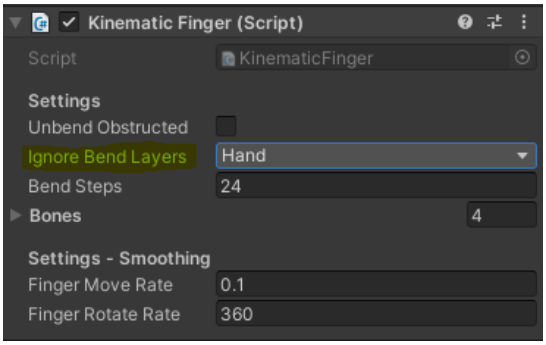




4. Navigate to the 'wrist' object in each of the hands hierarchies, select all of the top-most finger GameObjects in the 'Hierarchy' view.



5. In the 'Inspector' pane under the KinematicFinger component add your new add layer to 'Ignore Bend Layers'.

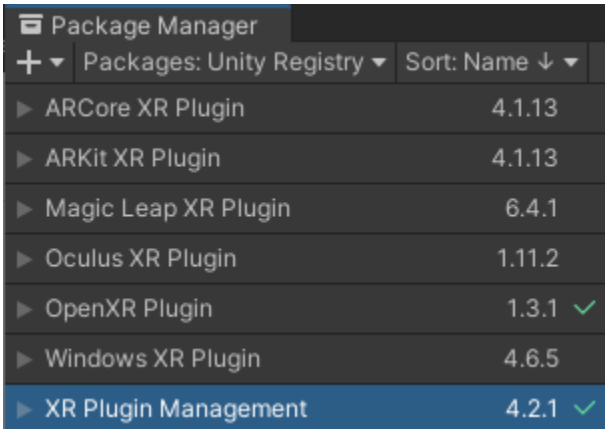


You have now finished this step! Your hand's fingers will bend properly without self-obstruction.

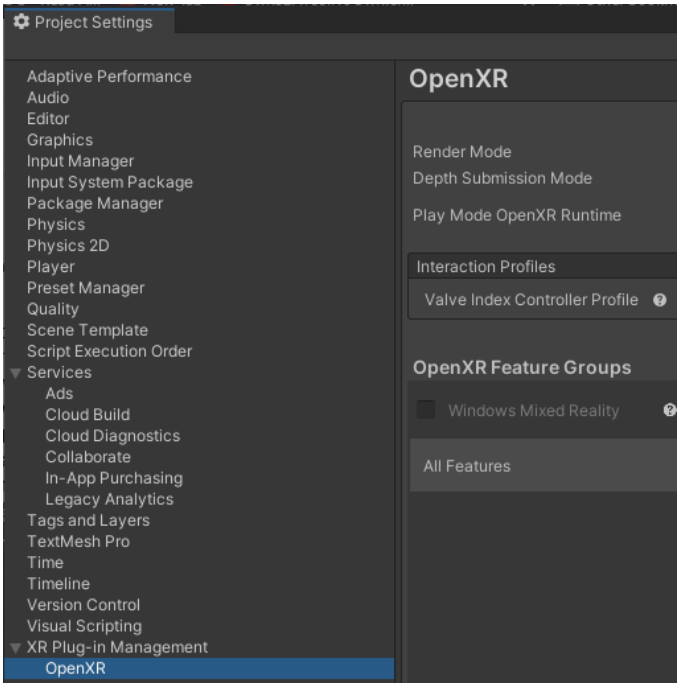
3.b. Setting Up VR In Your Project

3.b.i. Setting Up XR Plugin Management & OpenXR

- You may skip this step if VR is already configured in your project.
- Start a new (or open an existing) Unity VR project.
  - You can start a VR project using the 'New Project' feature in the Unity Hub.
  - You can ***upgrade an existing non-VR project*** by opening the 'Package Manager' window and installing the '***XR Plugin Management***' package from the 'Unity Registry'.
    - Install another package such as OpenXR (recommended), SteamVR, or similar.



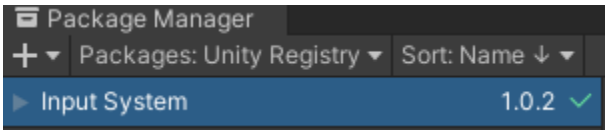
    - The last step to setting up 'XR Plugin Management' is to configure your chosen XR plugin – in the case of the screenshot below the 'OpenXR Plugin' was used.



- More detailed steps can be found on google regarding setting up VR in an existing Unity project.

3.b.ii. Import The New 'Input System'

- If you want to use the provided demo content make sure your project has the **new unity input system** enabled *(it may be enabled by default in newer versions of Unity)* so make sure you have the '**Input System**' package installed in your project if you want to use the provided demo content.
- Simply navigate to 'Windows → Package Manager' and import the 'Input System' package from the 'Unity Registry'.



You are now done 'getting started'.
Check out any of the demo scenes or start making your game!

The 'Physics Hand' Module

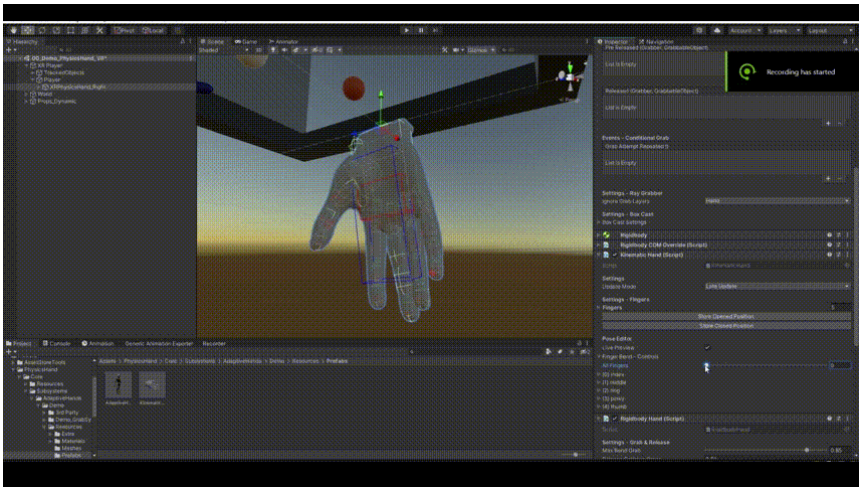
The module that ties all of the subsystems together to make a realistic physics hand..

4.a. The RigidbodyHand Component

4.a.i. The Component

- The RigidbodyHand component is the main component for free-hand style physics hands.

4.a.ii. Dynamic Grab Boxes & Visualization



a GIF image showing the dynamic grab box feature shrinking the grab area as the hand closes.

- The 'dynamic grab box' feature allows the relevant RayGrabber components grab box volume to be dynamically resized between two target values based on the 'RigidbodyHand.KinematicHand.AverageFingerBend' value.
- The red box represents the grab box while KinematicHand.AverageFingerBend is equal to 1.
- The blue box represents the grab box while KinematicHand.AverageFingerBend is equal to 0.
- The yellow box represents the grab box while KinematicHand.AverageFingerBend is at its current state.
- If you do not see the yellow box ensure the RayGrabber component is not collapsed in the 'Inspector' pane.
- **In order to see the live visualization of the yellow box you must have 'Live Preview' mode enabled.**
- If you do not see the blue and red boxes ensure your RigidbodyHand components GameObject is selected in the 'Hierarchy' pane and ensure the RigidbodyHand component is uncollapsed in the 'Inspector' pane.

4.b. The RigidbodyHandUIPointer Component

4.b.i. The Component

- The RigidbodyHandUIPointer component allows you to interact with default Unity UI elements seamlessly from a distance or up close.
- ***Only one*** RigidbodyHandUIPointer should be enabled at a time if you are using the 'takeoverCanvases' setting instead of manually assigning the UI pointers to specific canvases.
- RigidbodyHandUIPointers cannot operate on the same canvas at the same time.
- **Settings:**

Setting	***Description***	***Default Value***
UI Pointer	A reference to the Transform used to determine where to raycast from. (Raycasts happen in this transformed forward direction.)	null *(Transform)*
Auto Press UI Distance	The distance at which UI is auto pressed without the need to press the input action. (Use 'Infinity' to disable this.)	0.05f *(float)*
Max UI Distance	The maximum distance at which this pointer can interact with UI.	1f *(float)*
UI Layers	A LayerMask defining which layers the UI pointer can interact with.	Only 'UI' *(LayerMask)*
Ray Hit Triggers	Should this UI pointers physics-based (not EventSystem based) raycasts hit triggers?	true *(bool)*
Takeover Canvases	*IMPORTANT:* Should this pointer automatically take over all world space canvases when enabled? When true this component sets all world space canvases 'worldCamera' references to the UI pointer camera for the component, this means this UI pointer will have control over inputs to all world space canvases.	true *(bool)*

- The 'Visualization - Hit Marker' settings allow you to specify a hit marker on your UI pointer aim point.

Subsystem Documentation

The core systems that make up 'Physics Hand'.

5.a. Time System

- The module that provides the current time.
- Why does this module exist? This module exists to allow you to easily change what 'current time' retrieval method is used. This lets you use realtimeSinceStartup, unscaledTime, or time depending on what your game requires.
- API Reference: Time System - API Reference

5.b. Physics Tools

- A module that provides useful components and helper functions for many different collision-related, velocity tracking, rigidbody processing use cases and more.
- Documentation: Physics Tools - Documentation
- API Reference: Physics Tools - API Reference

5.c. Adaptive Hands

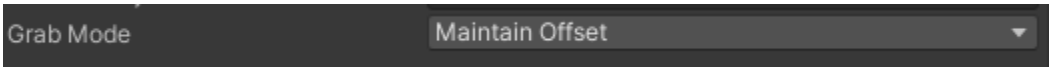
- The module that provides the visual kinematic hand control.
- Documentation: Adaptive Hands - Documentation
- API Reference: Adaptive Hands - API Reference

5.d. Grab System

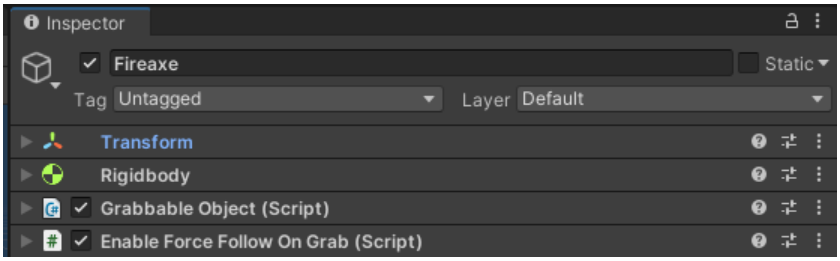
- The module that provides grabbing and throwing support.
- Documentation: Grab System - Documentation
- API Reference: Grab System - API Reference

5.d.i Making a 'Non-Stationary Maintain Offset' Grabbable

- An important thing to note about the implementation of 'Grab System' in 'Physics Hand' is that GrabbableObjects whose grab type are set to 'Maintain Offset' are not, by default, movable with the hand.
- There are many cases where you may want to have both the effects of 'Maintain Offset' where the physics hand perfectly maintains the offsets it grabbed the GrabbableObject at, for example long 2-handed grabbable objects.
- ***All you need to do to make a 'Non-Stationary Maintain Offset' GrabbableObject*** is:
    1. Add your GrabbableObject and Rigidbody to your GameObject that you want to be grabbable.
    2. Ensure your grabbable object has some Colliders to grab, if they are *nested in child* GameObjects make sure to add a GrabbableChildObject component to the same GameObject as the Collider you want to make grabbable.
    3. Make sure the GrabbableObject component you added (or are modifying) is using the 'Maintain Offset' grab mode.

    4. Finally, simply add a EnableForceFollowOnGrab component to the same GameObject as the GrabbableObject, this is the final step to (re)enable the physics hands movement forces for your 'Maintain Offset' grabbable.

- Check out the included 'Fireaxe' prefab in the 'Demo' folder for an example usage.

5.e. Editor Ghosts

- The module that creates and manages ghost representations of objects in the editor.
- API Reference: Editor Ghosts - API Reference

5.f. Haptics System

- The module that is responsible for haptic vibrations.
- API Reference: Haptics System - API Reference

5.g. PIDLibrary

- The module that is responsible for PID control systems.
- API Reference: PIDLibrary - API Reference

Extensions

Extensions provide additional, optional functionality to Physics Hand.
Extensions may depend on each other, for example 'PlaceSystem' depends on 'TypeReferences' for some components.

6.a. Type References

- The module that is responsible for serializing class types and displaying them in the editor.
- API Reference: Type References - API Reference

6.b. Place System

6.b.i. The PlacePoint Component

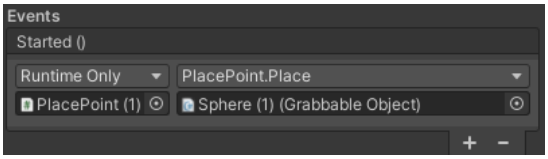
- Any kinematic or non-kinematic GrabbableObject can be placed into a PlacePoint.
- The 'Parent On Place' option makes it so any placed objects become a child of the place point making it so they move around with the place point. This is ideal for things such as holsters.
- The 'Deactive On Release' option makes it so when any object is released from the place point the place points gameObject becomes deactivated immediately.
- When would you not want a place pointing to 'Parent On Place'? Consider cases such as a minigame where the player has to drop objects into some area. In this case a non-parenting place point may be used.

6.b.ii. Starting With a GrabbableObject In a PlacePoint

- To start with a GrabbableObject already in a PlacePoint you simply need to use the 'Started' event in the relevant PlacePoint component (or any similar event in any component would work).
- Simply follow these steps:
    1. Click the '+' button in the 'Started' event to add an entry.
    2. Drag the place point you want the GrabbableObject to be placed in into the left-hand side object box.

A screenshot showing the left-hand side object box after dragging the PlacePoint into it.

    3. From the functions dropdown select 'PlacePoint.Place'.
    4. Drag the GrabbableObject that you want to be placed into the right-hand side object box.

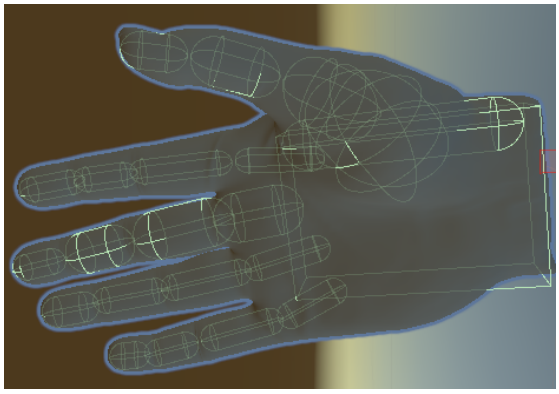


A screenshot showing the 'PlacePoint.Started' event being used to place a GrabbableObject in the PlacePoint when the game starts

Configuring Your Physics Hand

7.a. Setting Up The Colliders

- You will want to set up a collider that nicely fits both on the wrist bone *(or a child object if you want to rotate the collider)* and on each finger bone *(or a child object if you want to rotate the collider)* when you are done your hand should look something like the one in the screenshot below:

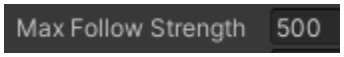


A screenshot showing the colliders (green) that make up the demo XRPhysicsHands collision model.

- Remember that the ***physics colliders are not the same as the 'Adaptive Hands' colliders***. The physics colliders interact with the physics world whereas the 'Adaptive Hands' colliders detect finger bend obstructions.
- **For maximum realism** make sure your fingers' ***physical colliders*** are ***slightly larger*** than the ***'Adaptive Hands' colliders***, this will ***allow your hands fingers to scoop objects towards the palm while grabbing***.

7.b. Fixing Hand Jitter

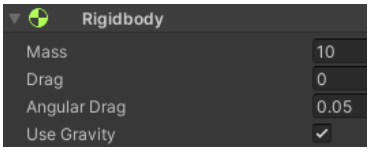
- Hand jitter is generally caused by one (or a combination) of the following things:
    a. The 'x, y, and/or z move PID' tuning may be incorrect in the ForceFollowTransform.
        i. Look up PID tuning for more details.
    b. The 'rotation PID' tuning may be incorrect in the ForceFollowTransform.
        i. Look up PID tuning for more details.
    c. The 'Max Follow Strength' being too high in the ForceFollowTransform component for the hand.



    d. The 'Max Follow Torque' being too high in the ForceFollowTransform component for the hand.

    e. The 'Max Follow Distance' of the ForceFollowTransform component is too low.
        i. 'Follow Strength' and 'Follow Torque' are scaled using the relevant 'Follow Strength (or Follow Torque) Curve' and the calculation 'follow distance / max follow distance'.

    f. The 'Angular Drag' on the hand's Rigidbody component is too low.
        i. The 'Angular Drag' on a Rigidbody acts as an angular velocity damping value .
        ii. This value can be 0 with a well tuned 'rotation PID'.
    g. The 'Drag' on the hand's Rigidbody component is too low.
        i. The 'Drag' on a Rigidbody acts as a velocity damping value.
        ii. This value can be 0 with a well tuned 'move PIDs'.
    h. The 'Mass' of the hand's Rigidbody component is too low.



Grab System Extensions

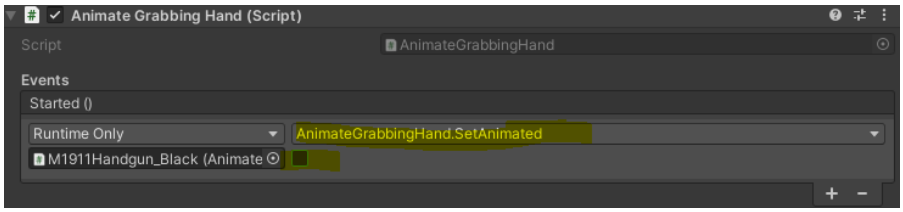
8.a. The ReleaseNocollideGrace Component

A screenshot of the ReleaseNocollideGrace component inspector in the Unity Editor.

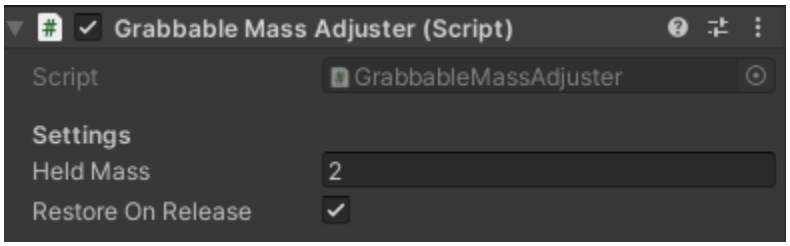
- Allows the 'release collision grace' setting to be overridden for specific GrabbableObjects by simply attaching this component to the same GameObject as them.
- Use the 'Release Collision Grace' setting to set the amount of time the GrabbableObject on the same GameObject will be no-collided with the releasing hand for on release.

8.b. The AnimateGrabbingHand Component

- The AnimateGrabbingHand component provides the following public methods:
  - AnimateGrabbingHand.Play(string pClip)
  - AnimateGrabbingHand.Trigger(string pTrigger)
  - AnimateGrabbingHand.EnableBool(string pName)
  - AnimateGrabbingHand.DisableBool(string pName)
  - ***The relevant SetBool, SetInteger, and SetFloat methods are also implemented however due to them having 2 arguments they will not be available in Unity editor event dropdowns are only available through the scripting API.***
- These methods allow you to play or trigger animations on any RigidbodyHand component that is currently grabbing the GrabbableObject that is on the same GameObject as the AnimateGrabbingHand component.
- The AnimateGrabbingHand.Animate boolean can be used to check if the component is currently overriding the KinematicHand behavior of RigidbodyHands that are currently grabbing the relevant grabbable.
- The AnimateGrabbingHand.SetAnimated(bool pAnimated) method can be used to change whether or not the component should override the behavior of the KinematicHand component associated with the relevant RigidbodyHands.
- By default the AnimateGrabbingHand.Animate boolean will be true, to change this default behavior it is recommended you use the 'Started' event in the component as shown in the screenshot below:



8.c. The GrabbableMassAdjuster Component



- The GrabbableMassAdjuster component is attached to the same GameObject as a GrabbableObject with a Rigidbody and allows you to set the mass of a grabbable object while it is being held. Optionally you can choose to have the component not restore the original mass of the grabbable too.
-

Setting	**Description**	**Default Value**
Held Mass	The mass to override the relevant GrabbableObject's Rigidbody's mass with while it is being grabbed.	0.5f *(float)*
Restore On Release	When true the original mass of the relevant Rigidbody will be restored when the grabbable object is released by all hands, otherwise the 'held mass' will remain indefinitely.	true *(bool)*

8.d. The EnableForceFollowOnGrab Component

- Forces a grabbing physics hand's 'force follower' enabled after grabbing the grabbable object this component is attached to. Useful for restoring 'force follow' forces after grabbing a 'Maintain Offset' mode grabbable object..
- ***Why does this component exist?*** Because by default when you grab a GrabbableObject set to 'Maintain Offset' mode the physics hand will by default stop using force to follow the controller. This component allows you to easily keep that force enabled when grabbing certain objects.
- Simply attach this component to the same GameObject as a GrabbableObject that would normally disable 'force following' to prevent it from being disabled.

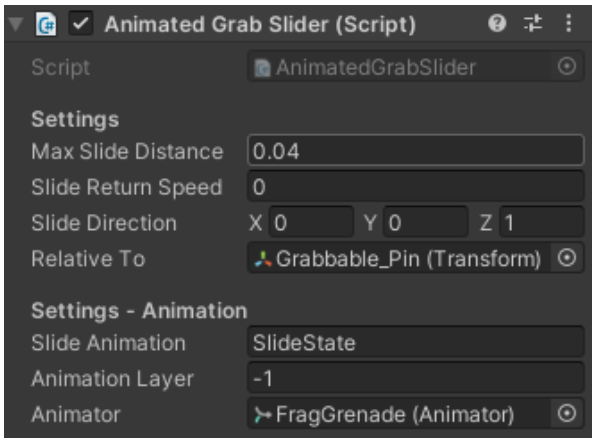
8.e. The RecloseHandOnGrab Component

- Forces a KinematicHand component that is attached to a Grabber that grabs the GrabbableObject that contains a RecloseHandOnGrab component to restart the hand closing process once the object is grabbed.
- This component is useful for components that are automatically grabbed through 'grab areas' or similar.
- A good example usage of this command can be found in the 'off hand grabbable' GrabbableObject in the XRWeapons demo on the demo handgun.

8.f. The ForceGrabWithinTrigger Component

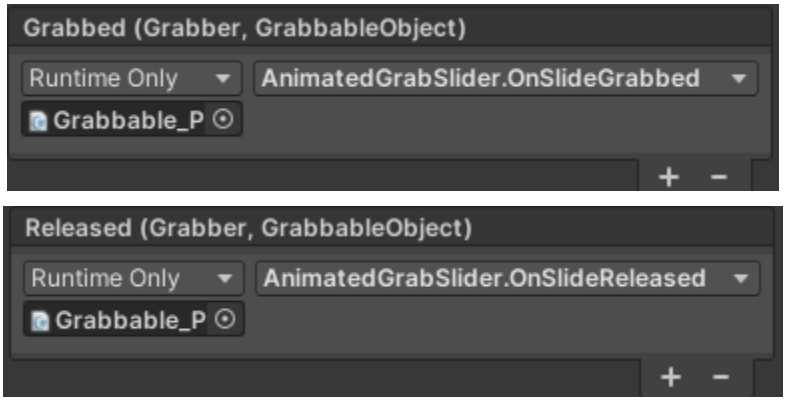
- A component that forces a Grabber/RigidbodyHand to grab an object while within the trigger, assuming one is not already being grabbed.

8.g. The AnimatedGrabSlider Component



A screenshot showing the AnimatedGrabSlider component 'Inspector' in the Unity Editor.

- A component that sets the normalized time of some animation based on the controllers distance since grab in some signed direction.
- The AnimatedGrabSlider component ***must be explicitly told using the provided OnSliderGrabbed(...) and OnSliderReleased(...) public methods when the slide is grabbed and released.*** To do this it is recommended to use the 'Grabbed' and 'Released' events from the GrabbableObject component.



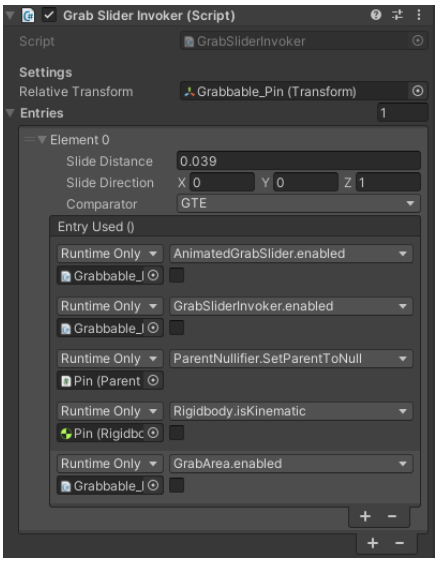
Screenshots showing the 'Grabbed' and 'Released' events being used to tell the AnimatedGrabSlider when the slide was grabbed and released.

- Settings:
-

Setting	**Description**	**Default Value**
Max Slide Distance	The maximum distance the slide can be slid along the slideDirection.	0f *(float)*
Slide Return Speed	The speed an ungrabbed slide returns in units per second. A value of 1 means in 1 second the slide would totally	0f *(float)*

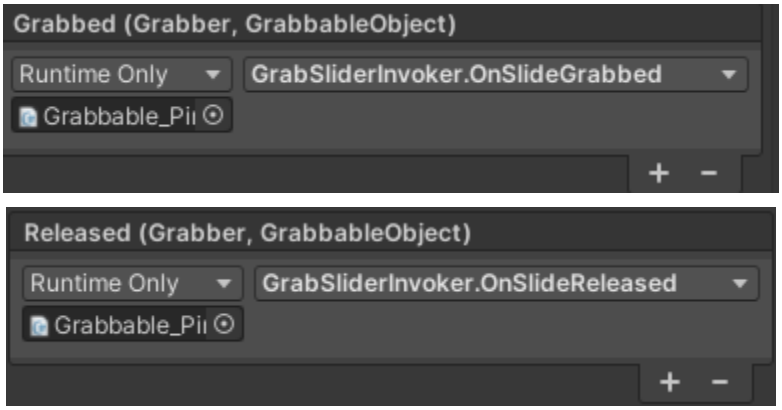
	return, a value of 0 means the slide would never return on its own.	
Slide Direction	A Vector3 specifying the direction the slide may be pulled in.	-Vector3.forward *(Vector3)*
Relative To	(Optional) A Transform that slide directions are considered relative to. **If null 'animator' will be used as a fallback, and 'transform' will be used as the final fallback.**	null, *(Transform)*
Animation Layer	The layer the slide animation is in. (-1 for default.)	-1 *(int)*
Slide Animation	The name of the animation state used for the slider.	"" *(string)*
Animator	A reference to the Animator where the slide animation exists.	null *(Animator)*

8.h. The GrabSliderInvoker Component



A screenshot of the 'Inspector' pane for the GrabSliderInvoker used for pulling the pin in the demo grenade.

- A component that allows you to specify signed distance 'slide' conditions that will invoke events.
- All slide distance conditions directions are considered relative to the 'RelativeTo' Transform property field. If none is specified in the settings of this component this value defaults to 'transform'.
- Multiple entries may have their conditions invoked in the same frame.
- This component is often used in conjunction with the AnimatedGrabSlider component.
- The GrabSliderInvoker component ***must be explicitly told using the provided OnSliderGrabbed(...) and OnSliderReleased(...) public methods when the slide is grabbed and released.*** To do this it is recommended to use the 'Grabbed' and 'Released' events from the GrabbableObject component.



Screenshots showing the 'Grabbed' and 'Released' events being used to tell the AnimatedGrabSlider when the slide was grabbed and released.

- Settings:

Setting	Description	Default Value
Relative Transform	A reference to the Transform that directions and distances are tested relative to. **If null this automatically defaults to 'transform'.**	null *(Transform)*
Entries	An array of GrabSliderInvoker.Entrys that have some conditions that may fire an event the first frame they are met since the start of the program or since the last time their condition was unmet.	null *(GrabSliderInvoker.Entry[])*

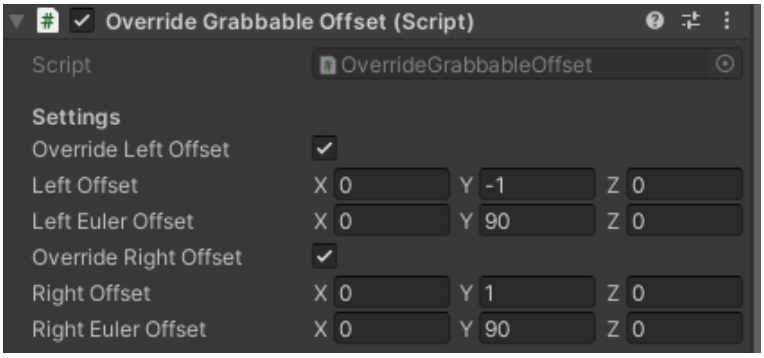
- This component is generally intended to be used in conjunction with Unity Editor events as seen in the 'FragGrenade' demo prefab included with the asset.
- Entry Structure:

Setting	Description	Default Value
Slide Distance	The target distance of the controller in the 'slideDirection' axis for the intended comparator comparison to trigger this entry.	0f *(float)*
Slide Direction	The direction the slide distance is measured in. Given in 'RelativeTo' local space.	Vector3.zero *(Vector3)*
Comparator	The FloatMath.Comparator to use when comparing the controller distance in the 'slideDirection' axis (left-hand side) to the 'slideDistance' value (right-hand side).	FloatMath.Compartor.GT *(FloatMath.Comparator)*
Entry Used()	An event that is invoked whenever the conditions for this entry are met after not being met previously.	N/A *(UnityEvent)*

8.i. The PhysicsGrabSyncer Component

- A component that ensures all Transforms are synced with the physics system before any key grab events occur. Gives perfect syncing at a slight performance cost during grabbing.
- This component is optional.
- To ensure perfect syncing with physics objects during grabs (especially when moving at high speeds) simply attach this component to the same GameObject as your RigidbodyHand component.

8.j. The OverrideGrabbableOffset Component



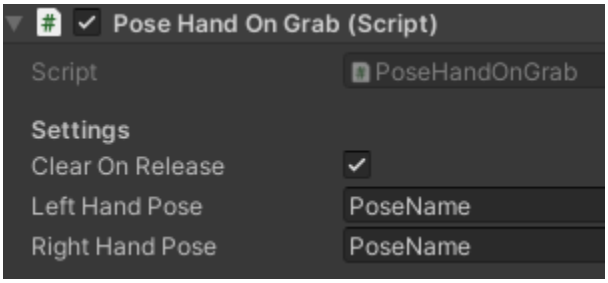
A screenshot of the OverrideGrabbableOffset Inspector pane in the Unity Editor.

- A component that overrides the local space offset and euler angle offset of a GrabbableObject that is on the same GameObject as this component that is grabbed while in 'move object' mode.
- This component is optional and is mainly useful when setting right/left grab offsets for an object with a pivot in an undesired spot.
-

Setting	**Description**	**Default Value**
Override Left Offset	A boolean that controls whether or not the left hand grabber should override the grab offsets.	true *(bool)*
Left Offset	A Vector3 that holds the offset to override the GrabbableObject's offset with when grabbed by a left hand. (In the local space of the grabber.)	Vector3.zero *(Vector3)*
Left Euler Offset	A Vector3 that holds the offset to override the GrabbableObject's euler angle offset with when grabbed by a left hand. (In the local space of the grabber.)	Vector3.zero *(Vector3)*
Override Right Offset	A boolean that controls whether or not the right hand grabber should override the grab offsets.	true *(bool)*
Right Offset	A Vector3 that holds the offset to override the GrabbableObject's offset with when grabbed by a right hand. (In the local space of	Vector3.zero *(Vector3)*

	the grabber.)	
Right Euler Offset	A Vector3 that holds the offset to override the GrabbableObject's euler angle offset with when grabbed by a right hand. (In the local space of the grabber.)	Vector3.zero *(Vector3)*

8.k. The PoseHandOnGrab Component

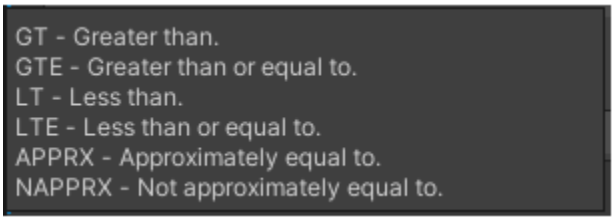


- A component that allows you to set a left or right hand's pose by name when it grabs a GrabbableObject on the same Gameobject as this component.
- The pose is cleared automatically when the object is released.

Adaptive Hands Extensions

9.a. The HandBendEvents Component

- The HandBendEvents component allows you to specify HandBendEvent.Entrys that lets you define a condition by giving an average bend target and a comparator to use.
- Conditions are checked as: <average finger bend of hand> <entry.comparator> <entry.averageBend>
- The first condition that passes its check will be used, if HandBendEvents.LastFiredEntry does not match the entry whose condition check was passed then the event 'Entry.EntryUsed' is invoked.
- ***Comparator descriptions**:*



- A demo usage of this component can be found in the provided 'XR Player' demo prefab on each of the hands. In the demo this component is used to change the UI pointers 'auto press' distance.

Extra 'Hand' Components

10.a. The HandCollisionEvent Component

- A simple component that invokes the relevant event whenever a RigidbodyHand component is involved in any collision event fired on this component.
- See the 'Demo_Button_Collision' demo prefab for example usage.
-

Event	Invoked By	Description	Argument(s)
CollisionEntered	OnCollisionEnter	Invoked whenever 'OnCollisionEnter' is invoked on this component for a collision involving a RigidbodyHand.	Arg0: Collision - the collision that took place.
CollisionStayed	OnCollisionStay	Invoked whenever 'OnCollisionStay' is invoked on this component for a collision involving a RigidbodyHand.	Arg0: Collision - the collision that took place.
CollisionExited	OnCollisionExit	Invoked whenever 'OnCollisionExit' is invoked on this component for a collision involving a RigidbodyHand.	Arg0: Collision - the collision that was taking place.

10.b. The HandTriggerEvent Component

- A simple component that invokes the relevant event whenever a RigidbodyHand component is involved in any trigger event fired on this component.
- See the 'Demo_Button_Trigger' demo prefab for example usage.
-

Event	Invoked By	Description	Argument(s)
TriggerEntered	OnTriggerEnter	Invoked whenever 'OnTriggerEnter' is invoked on this component by a Collider containing a RigidbodyHand.	Arg0: Collider - the collider that entered the trigger.
TriggerStayed	OnTriggerStay	Invoked whenever 'OnTriggerStay' is invoked on this component by a Collider containing a RigidbodyHand.	Arg0: Collider - the collider that stayed in the trigger.
TriggerExited	OnTriggerExit	Invoked whenever 'OnTriggerExit' is invoked on this component by a Collider containing a RigidbodyHand.	Arg0: Collider - the collider that left the trigger.

10.c. The IgnoreHandPoseAreaInGrab Component

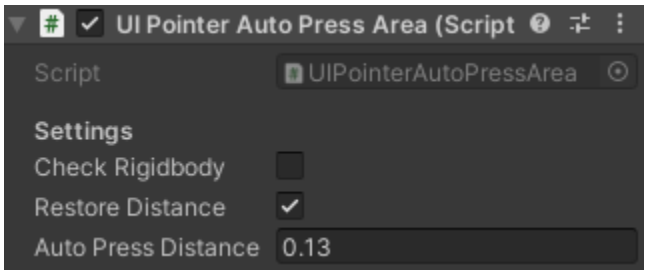
- Can be attached to the same GameObject as a HandPoseArea component to make it ignore hands with Grabbers that are currently grabbing.

10.d. The IgnoreBendStateAreaInGrab Component

- Can be attached to the same GameObject as a BendStateArea component to make it ignore hands with Grabbers that are currently grabbing.

Extra 'UI' Components

11.a. The UIPointerAutoPressArea Component



A screenshot showing the UIPointerAutoPressArea components 'Inspector' pane in the Unity Editor.

- A simple component that automatically overrides the 'Auto Press UI Distance' setting of RigidbodyHandUIPointer components that enter the trigger, and optionally restores the setting when they exit.
- This component automatically overrides any conflicting settings set by a HandBendEvents component, this is because by default in the example hand the HandBendEvents component is used to adjust the 'Auto Press UI Distance' of a same-gameObject RigidbodyHandUIPointer component. This prevents finger bend from overriding the RigidbodyHandUIPointer component's settings.
-

Setting	Description	Default Value
Check Rigidbody	Should this component check 'attachedRigidbody' to the Collider if no RigidbodyHandUIPointer is found on the collider directly?	false *(bool)*
Restore Distance	Should this component restore the 'Auto Press UI Distance' of pointers that leave the area?	true *(bool)*
Auto Press Distance	The 'Auto Press UI Distance' value to override the relevant setting within Rigidbody RigidbodyHandUIPointer	0.13 *(float)*

	components that trigger this.	

Developer Components

Components that are for development purposes and not intended for release builds.

12.a. The LogRelativeGrabOffset Component

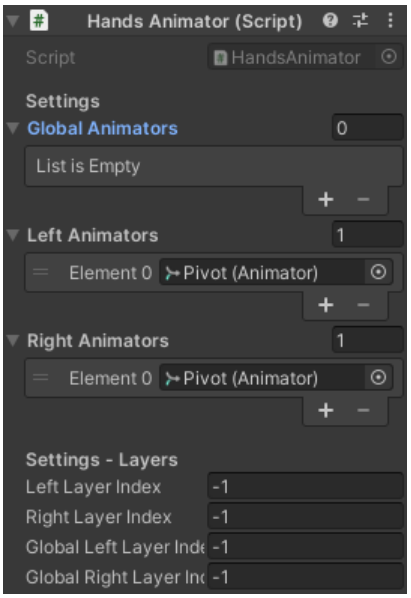
- Logs the relative position and euler angle offset of the GrabbableObject this component is attached to in local space of the Grabber that grabbed it at the time of grab and release. Useful to obtain offset information for advanced grab pose components, fixed grab placement, and similar.
- To use this component simply attach this component to the same GameObject as the GrabbableObject you want relative grab offsets for, launch your game, grab the object, then view the console to get the output.
- Useful for configuring the MaintainOffsetByRelativeAngle component.
- For documentation on the MaintainOffsetByRelativeAngle component see the 'Documentation.pdf for Grab System' document.

12.b. The LogProjectedGrabAngle Component

- Logs the Grabbers projected angle around some axis in the local space of the GrabbableObject this component is attached to.
- Useful for configuring the MaintainOffsetByProjectedAngle component.
- For documentation on the MaintainOffsetByProjectedAngle component see the 'Documentation.pdf for Grab System' document.

Animation Extensions

13.a. The HandsAnimator Component



A screenshot of the HandsAnimator component inspector in the Unity Editor.

- A component that can be used to apply animations to the right, left, or both hands uniquely or not. Capable of animating the hands of both humanoids and floating hands.
- This component is intended to be attached to a GameObject that is a parent of the relevant hands, or the same GameObject as the root of a humanoid.
-

Setting	**Description**	**Default Value**
Global Animators	An array of Animators that hand animations are 'globally' applied to. Both left and right hand animations are applied to them. Useful for animating the hands of humanoids.	null *(Animator[])*
Left Animators	An array of Animators that left hand animations are applied to.	null *(Animator[])*

	Useful for animating floating left hands.	
Right Animators	An array of Animators that right hand animations are applied to. Useful for animating floating right hands.	null *(Animator[])*
Left Layer Index	The index (or -1 for any) that left hand animations should be played on by 'left animators' of this component.	-1 *(int)*
Right Layer Index	The index (or -1 for any) that right hand animations should be played on by 'right animators' of this component.	-1 *(int)*
Global Left Layer Index	The index (or -1 for any) that left hand animations should be played on by 'global animators' of this component.	-1 *(int)*
Global Right Layer Index	The index (or -1 for any) that right hand animations should be played on by 'global animators' of this component.	-1 *(int)*

13.b. The AnimateHandOnGrab Component

A screenshot of the AnimateHandOnGrab component inspector in the Unity Editor.

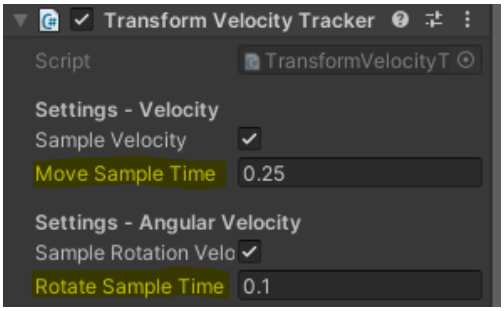
- A component that checks for a relevant HandsAnimator component when a hand grabs the relevant GrabbableObject and handles the playing of 'grab' animations.
- This component will play the 'left hand state' on 'left animators' and 'global animators' of the HandsAnimator when grabbed by a left hand.
- This component will play the 'right hand state' on 'right animators' and 'global animators' of the HandsAnimator when grabbed by a right hand.
- This component must be attached to the same GameObject as the relevant GrabbableObject.
- Check out this Youtube Tutorial on creating grabbable objects that force HandAnimators into a grab animation on grab.

FAQ

(Frequently Asked Questions)

Q: My hands and fingers won't bend!
A: Make sure you have completed the first step in '[Getting Started: Setting Up Layers]'.
For custom hands: Make sure you have stored an opened and closed hand position.

Q: How can I tune throwing to perform better at specific throw styles such as basketball throws?
A: Aside from throw strength (which can be adjusted through the GrabbableObject component or the RayGrabber component) the best way to tune throwing is to modify the *'Move Sample Time'* and the *'Rotate Sample Time'* settings in the TransformVelocityTracker component. Lower values work better for basketball style shots that rely on the flick of the wrist whereas higher values (to a point) work better for slow loopy throws.



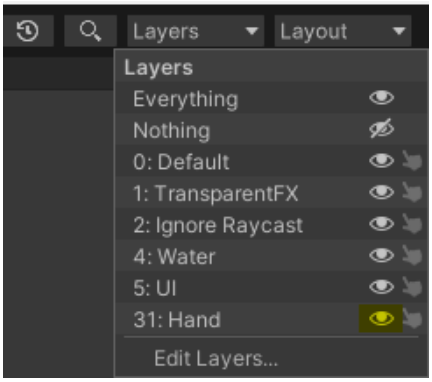
Q: After updating the 'Mesh_Hands.fbx" mesh my hands are invisible! How can I fix this?
A: Unfortunately Unity does not handle updating the SkinnedMeshRenderer for mesh instances where you have used the 'Unpack Completely' feature to unpack them. In these cases you must fix your prefab manually by re-adding the new, non-unpacked mesh to your prefab. Here is a youtube tutorial showing the steps to fix it.

Q: I can't edit the 'box cast data' for my RayGrabber component, why?
Whenever I edit the 'box cast data' for my RayGrabber component it resets.

A: This happens when previewing the RigidbodyHand component, to edit this data either temporarily disable the drawing of gizmos using the 'Gizmos' icon in the editor, or alternatively collapse the RigidbodyHand inspector to prevent it from overriding this. This feature overrides the box cast data to allow editor-time visualizations of the dynamic grab box at different 'live previewed' hand bend states.

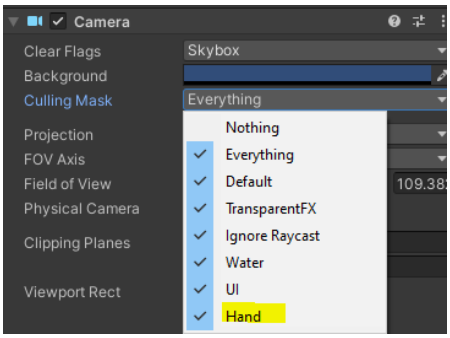
Q: I can't bend my hands fingers in 'Live Preview' mode. Even in the prefab editor.
A: This happens when some Collider from your scene is physically blocking the fingers from bending. It may be an invisible collider such as a BoxCollider from the ground in your scene. *So why does this still happen in the prefab editor?* When we test for collisions in edit mode, even in the prefab editor, it seems that collisions with scene objects are still detected. Try disabling whatever object the prefab is colliding with in your scene, ***or alternatively,*** you can simply load a blank scene before re-entering the prefab editor.

Q: The XR hands are not visible in the Unity Editor or in game.
A: Ensure in the Editor you do not have rendering disabled for the hand layer, the 'Layers' dropdown in the top right has options for in-editor layer visibility.



A screenshot showing the 'Layers' dropdown in the Unity Editor with the visibility toggle selected.

- If the hands are not rendering in game, ensure that your Camera's 'Culling Mask' is not culling the 'Hand' layer.

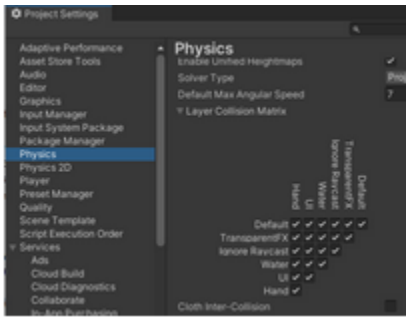


A screenshot showing the 'Culling Mask' dropdown in the Unity Editor 'Inspector' pane for a Camera component.

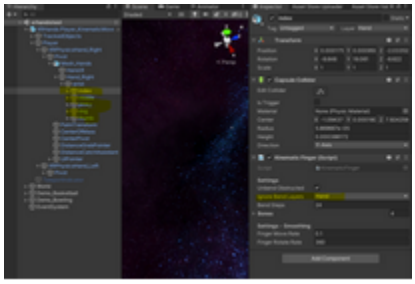
Q: My fingers won't bend, I can't grab things, and/or the Physics Hand is spazzing out/not reaching the target.

A: These issues occur when Colliders in your scene are blocking the Physics Hand and are not a bug, there are plenty of work-arounds that are easy to implement using 'Layers'. Here is some information to help understand the various work-arounds:

- ***Collision Layer Rules are for actual physics collisions.*** If the Physics Hands are being physically obstructed from reaching their target, fix this using Unity's 'Layers' and 'Layer Collision Settings' as shown in the screenshot below.



- ***Finger Bending uses layers with their own 'Ignore Bend Layers' (on each finger)*** setting to set collision layers to be 'ignored' when it comes to finger-bend obstructions. Objects in layers that are added to the 'Ignore Bend Layers' setting will not block the hand's fingers from bending.



- ***The RayGrabber component (on each hand) uses its own 'Ignore Grab Layers' too.*** Select each of your physics hands and add your ignored layer(s) to the 'Ignore Grab Layers' setting of your relevant Grabber component (likely RayGrabber). Any colliders in these ignored layers will no longer interfere with grabbing.

Q: My hand mesh is frozen in place on the Universal Render Pipeline when grabbing an object.

A: This issue is caused by the RigidbodyStabilizer component. The component was designed to increase hand stability when working with the built-in render pipeline.
Prior to v1.1.2 this component caused issues on scriptable render pipelines like the URP.
To fix this either simply disable the component using the checkbox in the editor or update to v1.1.2 or newer.